```
  end

  test "normal job raising a non-transient exception" do
    job_instance = CompleteOrderJob.new
    non_existent_order_id = -99
    CompleteOrderJob.perform_async(non_existent_order_id)
    assert_raises ActiveRecord::RecordNotFound do
      Sidekiq::Job.drain_all
    end
  end
  test "normal job raising a transient exception" do
    @payments_service_status.update(sleep: 0, throttle: false, crash: true)
    order = FactoryBot.create(:order)
    CompleteOrderJob.perform_async(order.id)
    assert_raises IgnorableExceptionSinceSidekiqWillRetry do
      Sidekiq::Job.drain_all
    end
  end
  class CustomTransientJob < ApplicationJob
    def self.transient_exceptions
      [
        ArgumentError
      ]
    end
    def perform(throw_argument_error)
      if throw_argument_error
        raise ArgumentError
      end
    end
  end
  test "job raising a custom transient exception" do
    order = FactoryBot.create(:order)
    CustomTransientJob.perform_async(true)
    assert_raises IgnorableExceptionSinceSidekiqWillRetry do
      Sidekiq::Job.drain_all
    end
  end
end
```

Because the test uses `CompleteOrderJob`, you can remove the explicit exception handling from it:

snapshots/4-4/sidekiq-book/app/jobs/complete_order_job.rb